TEST DOCUMENT · VERSION 2.0



PDFAR

Comprehensive PDF test document

A single fixture for layout, navigation, rendering and interaction: images, seven table styles, a five-level table of contents, mixed page sizes, a rotated page, annotations, form fields and embedded attachments.

Images · Tables · TOC H1-H5 · Annotations · Forms · Attachments · A3 · Rotate 90°

Generated 23 October 2026 · ReportLab + PyMuPDF · GPL-3.0-or-later

github.com/wachin/PDF-Advanced-Reader

Document Information

This file is a deliberately dense PDF fixture. It exists so every part of PDFAR can be exercised on one document: the continuous page layout, the tile renderer, text selection, search, the outline tree, the annotation layer, form widgets and the attachments panel. Nothing here is a real report; the content is synthetic, but the PDF structures are real and intentionally varied.

Property	Value
File name	demo.pdf
Title	PDFAR — Comprehensive PDF Test Document
Author	Washington Indacochea Delgado
Version	2.0 (fixture revision 2)
Generated	23 October 2026
Page sizes	A4 portrait, one A3 spread, one /Rotate 90 page
Structure	5-level outline, printed TOC, internal and external links
License	GPL-3.0-or-later

What This Document Exercises

Area	What to check	Section
Typography	H1–H5, inline styles, lists, quotes, code, two columns	2
Images	PNG with alpha, JPEG, grayscale, diagrams, 200 DPI scan	3
Tables	Grids, zebra rows, spans, long tables, rich and nested cells	4
Navigation	Outline depth, TOC links, URI links, page labels	5
Annotations	Markup, notes, free text, shapes, ink, stamp, file	6
Forms	Text, multiline, check, radio, combo, list, button	7
Attachments	CSV, ODS, JSON and TXT embedded files	8
Rendering	Transparency, gradients, A3, rotation, dense type	9

How to Use This File

1. Scroll from the cover to the appendix and watch for layout jumps.
2. Zoom with the toolbar, Ctrl+wheel and the 50–400% preset list.
3. Open the outline sidebar and expand entries down to level 5.
4. Search for **tile**, **annotation** or **reflow** to test hits.
5. Use **Tools > Attachments** (Ctrl+Shift+A) for embedded files.
6. Tab through the form fields in section 7 and toggle the widgets.
7. Select and copy text from the scanned page; it has a real text layer.

Table of Contents

The printed contents is generated from the same heading flowables that produce the PDF outline, so page numbers and hierarchy always agree. Level 1 is bold; levels 2–5 are progressively indented.

Document Information . . . 2
What This Document Exercises . . . 2
How to Use This File . . . 2
1. Introduction . . . 5
1.1 Purpose . . . 5
1.1.1 Intended Audience . . . 5
1.1.1.1 Before You Begin . . . 5
1.1.1.1.1 Conventions Used in This Document . . . 5
1.2 Feature Overview . . . 5
1.2.1 Architecture . . . 6
1.2.1.1 Rendering Pipeline . . . 6
1.2.1.1.1 Tile Lifecycle . . . 6
2. Text and Typography . . . 8
2.1 Paragraph Styles . . . 8
2.2 Lists . . . 8
2.2.1 Definition-Style Data . . . 8
2.3 Block Quotes and Code . . . 9
2.4 Two-Column Layout . . . 9
2.5 Unicode and Special Characters . . . 11
3. Images and Graphics . . . 12
3.1 Raster Images . . . 12
3.1.1 Diagrams . . . 12
3.2 Colour and Transparency . . . 13
3.3 Feature Overview Diagram . . . 15
3.4 Zoom Levels . . . 15
3.5 High-Resolution Scan . . . 16
3.6 Image Formats in This File . . . 17
4. Tables . . . 18
4.1 Simple Grid . . . 18
4.2 Zebra Rows and Alignment . . . 18
4.3 Merged Cells . . . 18
4.4 Long Table with Repeated Header . . . 19
4.5 Numeric Alignment and Units . . . 21
4.6 Rich Cell Content . . . 21
4.7 Comparison Matrix . . . 21
5. Navigation and Links . . . 22
5.1 Outline . . . 22
5.2 Internal Links . . . 22
5.3 External Links . . . 22
5.4 Page Labels . . . 22
6. Annotations . . . 24
6.1 Text Markup . . . 24
6.2 Notes and Comments . . . 24

6.3 Shapes and Freehand . . . 24
6.4 Annotation Inventory . . . 25
6.5 Rendering Notes . . . 26
7. Interactive Forms . . . 27
7.1 Text Fields . . . 27
7.2 Check Boxes and Radio Buttons . . . 27
7.3 Choice Fields . . . 27
7.4 Buttons . . . 28
7.5 Field Inventory . . . 28
8. Attachments and Embedded Files . . . 29
8.1 Embedded Payloads . . . 29
8.2 Why the Tables Are Native PDF . . . 29
8.3 Attachments in the Reader . . . 29
9. Rendering Challenges . . . 30
9.1 Transparency and Gradients . . . 30
9.2 Dense Text and Small Type . . . 30
9.3 Large-Format Page (A3) . . . 32
9.4 Rotated Page . . . 33
9.5 Mixed Content . . . 35
10. Appendix . . . A-1
10.1 Testing Checklist . . . A-1
10.1.1 Rendering . . . A-1
10.1.1.1 Scrolling . . . A-1
10.1.1.1.1 Tile Boundaries . . . A-1
10.1.2 Interaction . . . A-1
10.2 Glossary . . . A-2
10.3 Revision History . . . A-2
10.4 Colophon . . . A-2

1. Introduction

PDFAR is a Linux-native reader built around PyMuPDF for parsing and rasterization and PyQt6 for the interface. Its design goal is simple: large documents should open quickly, scroll smoothly and remain searchable, selectable and navigable. This document is the counterpart to that goal — a PDF that is intentionally awkward in all the places a viewer is likely to be awkward too.

1.1 Purpose

The fixture covers page geometry, text layout, raster and vector graphics, table structures, annotation dictionaries, AcroForm widgets, embedded files and a five-level outline. It also includes a small number of deliberate stress cases: a full-page 200 DPI scan, a transparent PNG, an A3 page inside an A4 document and a page whose orientation is encoded with a **/Rotate 90** entry rather than a landscape media box.

1.1.1 Intended Audience

The document is useful to anyone testing a PDF viewer, but it is primarily aimed at PDFAR contributors. If a change touches rendering, navigation or the annotation layer, open this file before and after the change and compare the same pages. The section order is stable; the page numbers may move when content is added.

1.1.1.1 Before You Begin

Use a build with the optional attachments panel available. The fixture assumes that the renderer honours the page tree, the outline and the AcroForm dictionary. A viewer that ignores annotations will still render the pages, but sections 6 and 7 will look empty.

1.1.1.1.1 Conventions Used in This Document

Keyboard shortcuts appear as **Ctrl+F**. Literal values appear in `monospace`. Links are blue and clickable. Measurements use points (1 pt = 1/72 inch) and percentages of the page size. A superscript marker such as “tile[1]” refers to the notes at the end of this introduction.

[1] A tile is a rectangular render unit; PDFAR chooses a tile size large enough to amortize PDF interpretation but small enough to keep memory bounded.

1.2 Feature Overview

The pages that follow are organised by feature area rather than by implementation layer. Each section ends with enough material to make the behaviour visible: for example, the table section contains a long table whose header repeats on the next page, and the annotation section places every supported markup annotation on a real text line.

- **Layout:** continuous scrolling, fit width, fit page and additive zoom steps.
- **Text:** selection, search with AND/OR/phrase semantics and regex search.
- **Reading:** reflow mode with adjustable text size.
- **Navigation:** thumbnails, five-level outline and internal links.
- **Interaction:** annotations, forms and embedded attachments.

1.2.1 Architecture

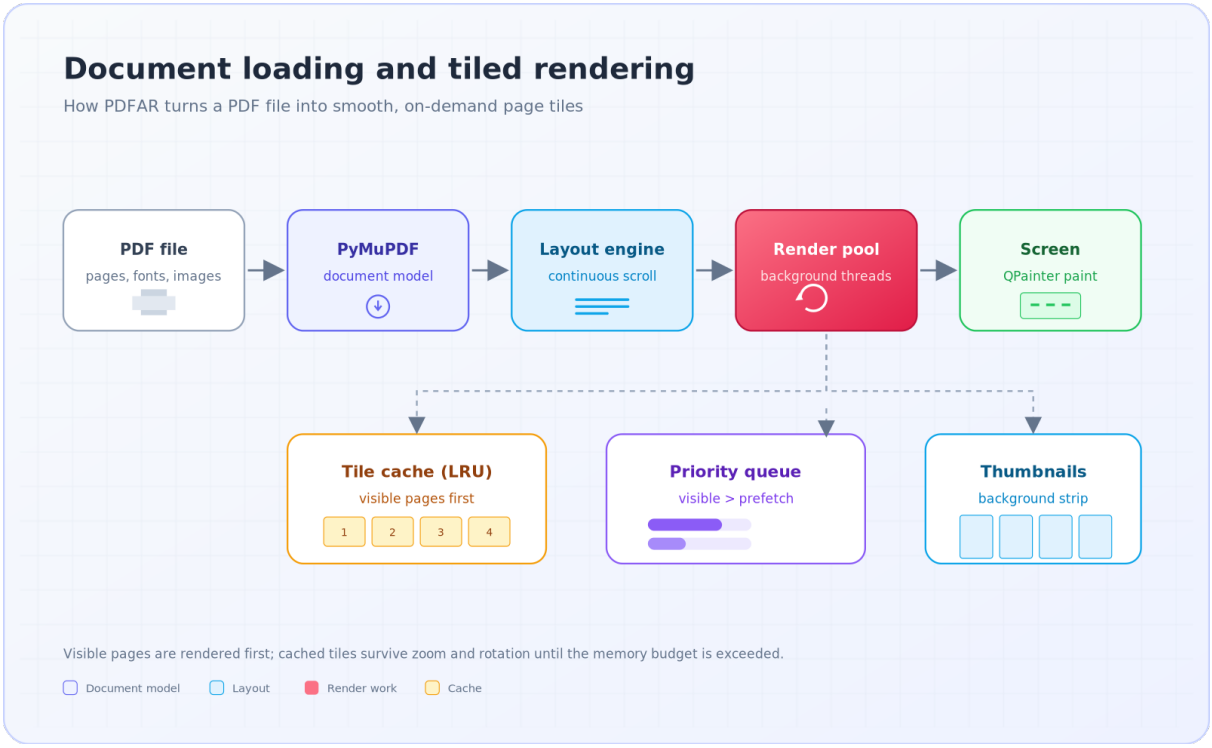


Figure 1.1 — Document loading and tiled rendering. Visible pages are prioritised over prefetch work.

1.2.1.1 Rendering Pipeline

Rendering starts when the view asks for a page rectangle. The render pool looks up cached tiles first; on a miss it schedules a background job, and the view paints a low-resolution placeholder until the tile arrives. The pipeline is deliberately split into five stages so that a slow page never blocks the Qt event loop.

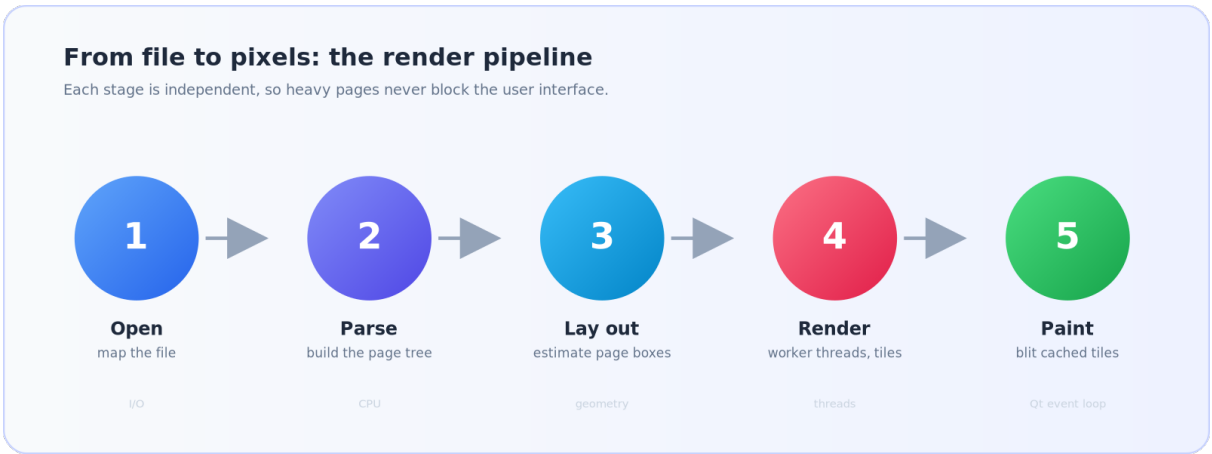


Figure 1.2 — The five render stages. Stage 4 runs on worker threads; the other stages are cheap.

1.2.1.1.1 Tile Lifecycle

Each tile moves through a small state machine. The table below is also a compact test of header styling, cell padding and zebra fills.

State	Entered when	Next state
Requested	The view needs the tile and the cache misses	Rendering

Rendering	A worker starts interpreting the page	Ready / Failed
Ready	The pixmap is available and uploaded	Cached
Cached	The tile fits the memory budget	Evicted
Evicted	The LRU budget is exceeded	Requested

2. Text and Typography

Text is the hardest part of a PDF to render consistently: fonts may be subset or missing, glyphs may be positioned individually, and reading order may differ from drawing order. The following pages use embedded DejaVu fonts so that the same glyphs are available to the renderer and to text extraction.

2.1 Paragraph Styles

This paragraph is justified and uses the serif body face at 10.1 pt. Inline styles include **bold**, *italic*, ***bold italic***, coloured text, `monospace fragments`, H_2O and x^2. The line spacing is fixed so that the vertical rhythm is predictable when the text is reflowed.

A second paragraph checks hyphenation-free justification with a longer measure. Justified text on a 487 pt measure produces modest word spacing; if the renderer applies synthetic letter spacing, the difference is visible at 200% zoom. The fixture also contains a narrow two-column section later in this chapter, where the measure is roughly half as wide.

A third paragraph mixes inline code with prose: call `view.set_zoom(1.25, anchor=pos)` to keep the point under the cursor stable. The anchor is expressed in view coordinates, not page coordinates, which is the detail that makes wheel zoom feel natural rather than jumpy.

2.2 Lists

Bulleted lists use a compact bullet and a hanging indent:

- Open a document and verify that the first page appears before the thumbnails finish.
- Press **PgDn** repeatedly and confirm that scrolling remains smooth across page boundaries.
- Toggle the sidebar and check that the document view keeps its scroll position.
- Rotate the page twice and confirm that text selection still follows the glyphs.

Ordered lists are numbered explicitly; the fixture uses 1–4 to avoid ambiguity:

1. Load `demo.pdf` from the repository root.
2. Open the outline and expand section 10 to level 5.
3. Use **Ctrl+F** to search for *annotation*.
4. Open **Tools > Attachments** and inspect the five embedded files.

2.2.1 Definition-Style Data

Term	Meaning in this fixture
Tile	Rectangular render unit cached independently of the page.
Anchor	View point held fixed while zooming.
Reflow	Single-column text mode that ignores the page geometry.
Widget	An AcroForm field with a rectangle and a value.

2.3 Block Quotes and Code

> *The goal is not to imitate a word processor. The goal is to make a structurally rich PDF feel as direct as a plain text file: scroll, select, search, annotate.*

```
def on_viewport_changed(self, rect):
    visible = self.tiles.intersecting(rect)
    for tile in visible:
        self.pool.request(tile, priority=HIGH)
    self.pool.trim(self.cache.budget_bytes)
```

Note
The code block is drawn as text, not as an image. It should be selectable, searchable and copyable — including the indentation.

2.4 Two-Column Layout

This page switches to a two-frame page template. Frames are filled in order, so the text flows down the left column and continues at the top of the right column. A viewer that reflows the page into one column should still preserve the reading order.

The left column begins with a short introduction. The frame width is 232 pt, which is narrow enough to expose excessive word spacing in justified text. The renderer should keep the text inside the frame and break lines at word boundaries.

The second paragraph continues the test with a mix of long and short words. A wide word such as “interoperability” must not overflow the column edge; if it does, the overflow is a useful regression signal.

The right column starts after the left frame is full. The two frames share the same baseline grid, so paragraphs should line up horizontally even though they are independent flowables.

Finally, this sentence is intentionally long so that the last line of the right column ends near the frame boundary. Selection across the column break should follow reading order, not drawing order.

2.5 Unicode and Special Characters

The body font is embedded as aTrueType subset. The samples below cover Latin accents, Greek, Cyrillic, currency, mathematical operators and arrows. Missing glyphs usually appear as boxes; the fixture makes that failure obvious.

Script	Sample	Notes
Latin accents	áéíóú ñ ç ü ø å œ	Western European coverage
Greek	αβγδε ζηθικ λμνξο πράσινο	Words plus individual letters
Cyrillic	Привет, мир! Документ PDF	Russian sample
Math	∑ ∏ √ ∞ ≈ ≠ ≤ ≥ ± × ÷	Symbols used in formulas
Arrows	← ↑ → ↓ ↔ ⇒ ⇐ ⇔	Navigation notation
Currency	€ £ ¥ ₹ ₽ ₩ ¢	Mixed currency symbols

Ligatures are intentionally avoided in the embedded subset: the text should round-trip through copy and paste as the original code points. The em dash — and the ellipsis … are present for punctuation tests.

3. Images and Graphics

The fixture mixes raster and vector content at several resolutions. The 200 DPI scan page is the heaviest object in the file; the diagrams are rasterised from SVG once and reused at display size.

3.1 Raster Images

The application icon is a PNG with an alpha channel. The transparent corners must show the page background, not a black or white square.



Figure 3.1 — Program icon rendered at 72 pt from a 512 px PNG with alpha.

3.1.1 Diagrams

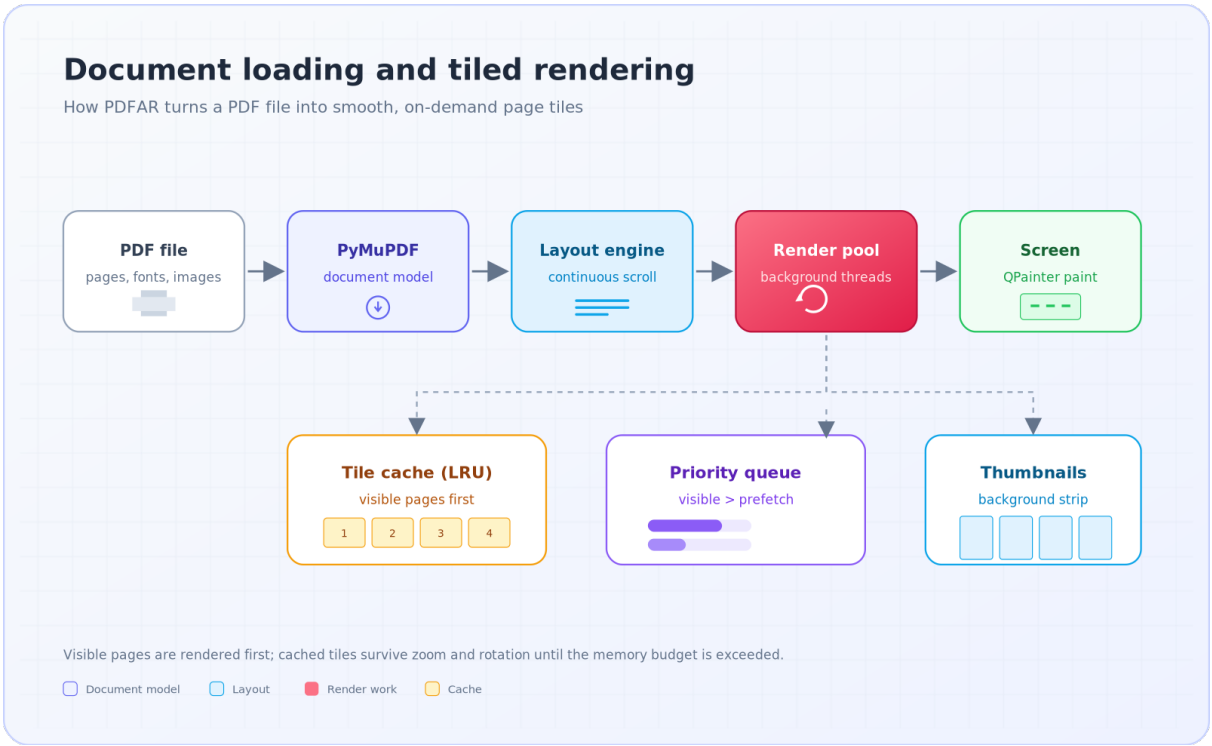


Figure 3.2 — Architecture diagram rasterised at 1600 px wide and placed at 452 pt.

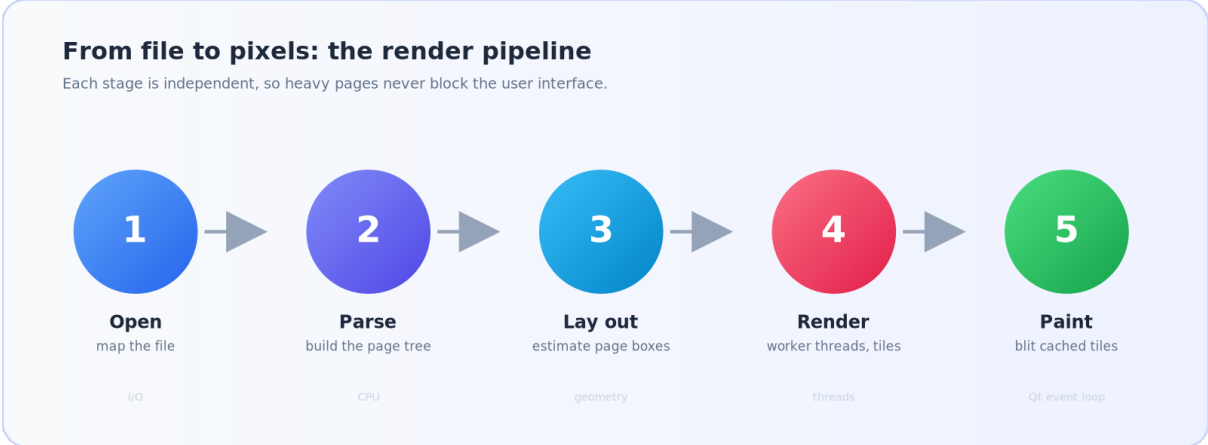


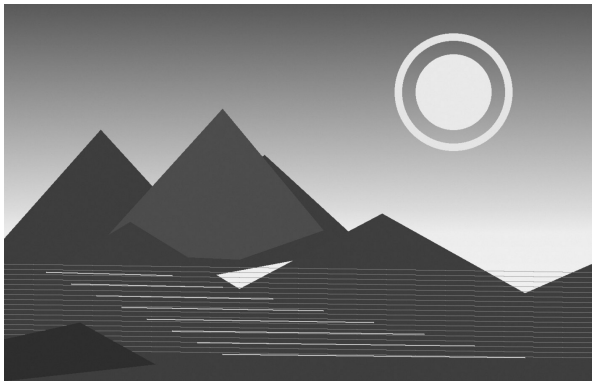
Figure 3.3 — Render pipeline. The source is a vector SVG; the PDF embeds a high-resolution PNG.

3.2 Colour and Transparency

JPEG and PNG exercise different decode paths. The photograph on the left is full colour; the image on the right is a grayscale JPEG with the same dimensions. Both are placed at 220 pt wide.



Colour JPEG · 1400×900



Grayscale JPEG · DeviceGray

The next image is an RGBA PNG placed over a coloured background. The semi-transparent shapes must blend with the band, and the text inside the image must remain crisp.

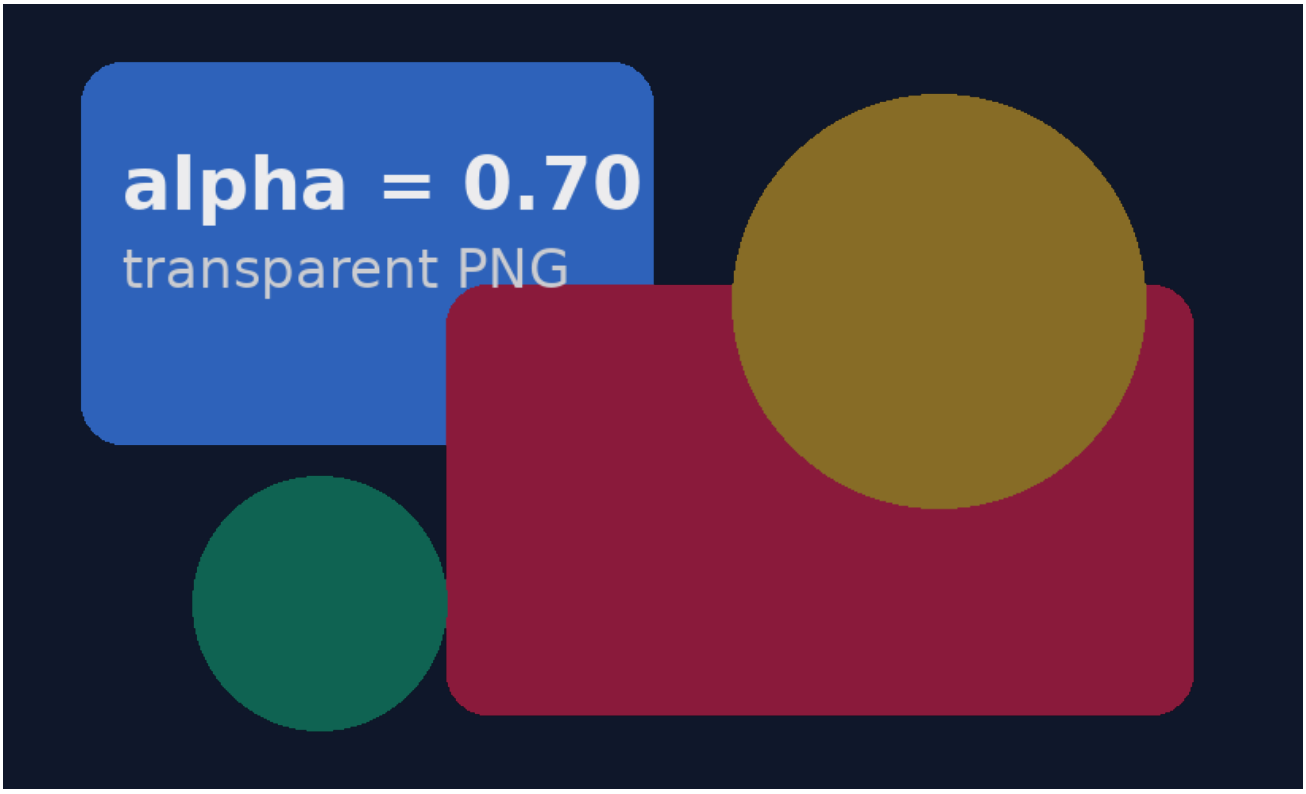


Figure 3.4 — Transparent PNG over a dark band.

3.3 Feature Overview Diagram

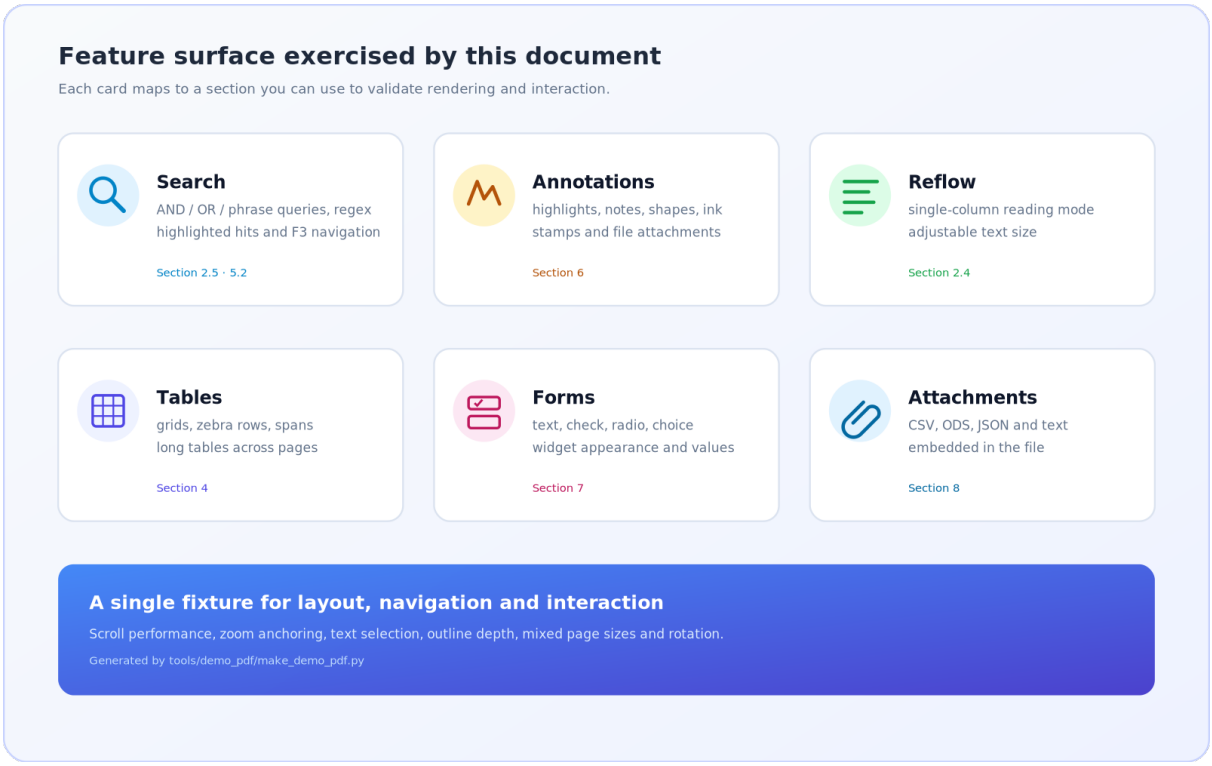


Figure 3.5 — Six feature areas and the sections that exercise them.

3.4 Zoom Levels



Figure 3.6 — Additive zoom levels. Each step adds a fixed percentage instead of multiplying the current scale.

3.5 High-Resolution Scan

The raster below is a synthetic 200 DPI scan: a single large JPEG with paper grain, a table, a stamp and a text-like page layout. The PDF also carries a real text layer on the page so search and selection can be tested against an image-heavy page.

The image is placed at 385 × 544 pt. At 100% zoom the viewer is downscaling a 1654 × 2338 JPEG, which is exactly the case where naïve renderers become blurry or slow. The caption is vector text separate from the image.

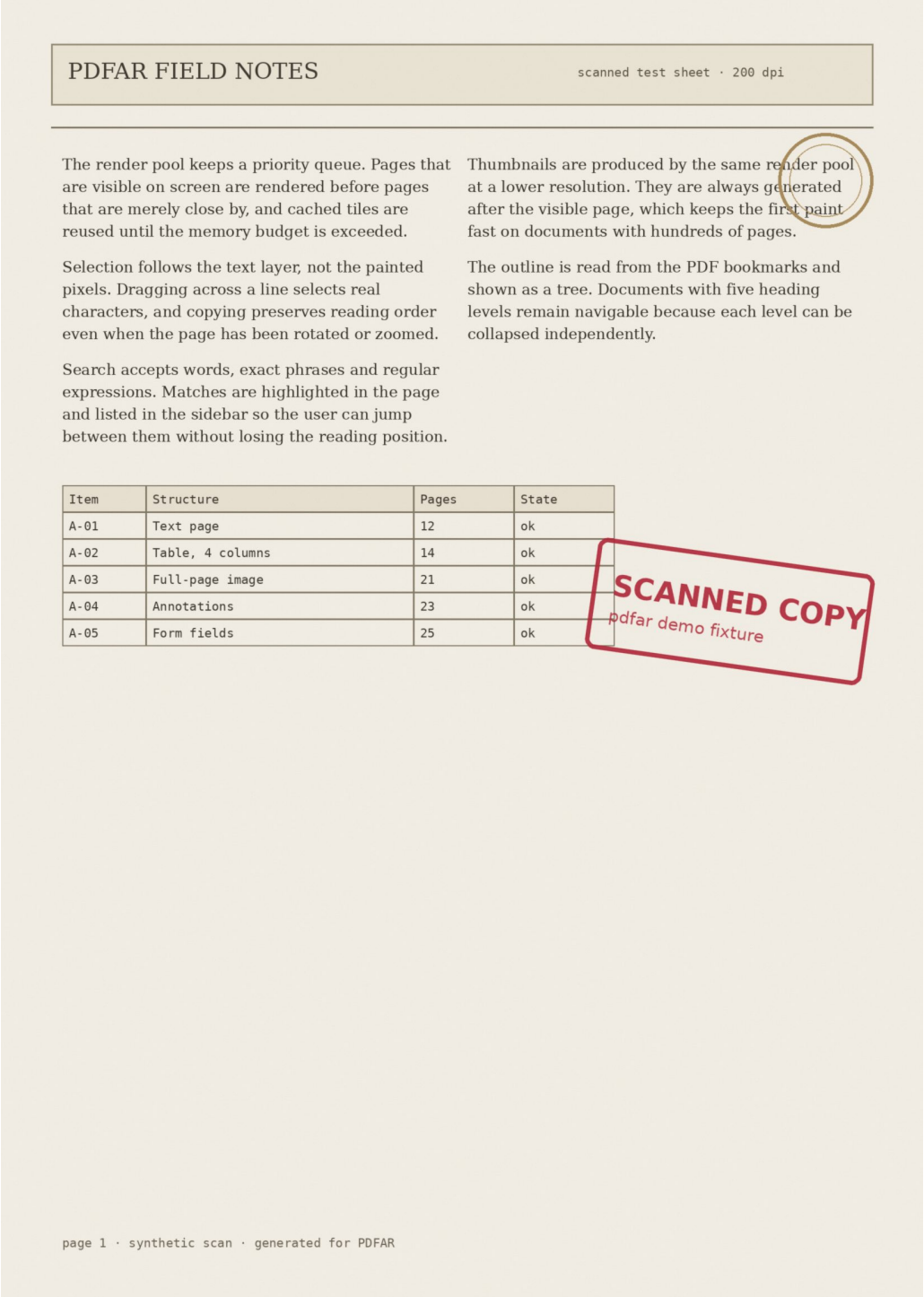


Figure 3.7 — Synthetic scanned page, 200 DPI, embedded as JPEG.

3.6 Image Formats in This File

Object	Format	Colour space	Alpha	Purpose
Icon	PNG	RGB	Yes	Transparent corners
Architecture	PNG	RGB	No	Large diagram
Pipeline	PNG	RGB	No	Wide diagram
Photograph	JPEG	YCbCr	No	Full colour
Grayscale	JPEG	Gray	No	DeviceGray path
Overlay	PNG	RGBA	Yes	Alpha blending
Scan	JPEG	YCbCr	No	Heavy page

4. Tables

Tables are the densest test of a PDF layout engine: they combine grid lines, background fills, wrapped paragraphs, alignment, row spans and page breaks. The seven tables below are generated natively in the PDF so their text remains selectable and searchable. The same data is also embedded as a CSV and an ODS workbook in section 8.

4.1 Simple Grid

Structure	Where used	Selectable	Notes
Paragraph	Body text	Yes	Justified measure
Heading	Outline and TOC	Yes	Levels 1–5
Table	This section	Yes	Grid and fills
Image	Section 3	No	Raster content
Annotation	Section 6	Partly	Text in popups

4.2 Zebra Rows and Alignment

Page	Size	Objects	Render ms	Cache	State
1	A4	12	18	2	warm
2	A4	9	14	2	warm
3	A4	15	22	3	hot
9	A4	7	11	1	cold
12	A4	1 image	48	6	hot
36	A3	38	61	8	cold
37	A4 /Rotate	6	19	2	warm

4.3 Merged Cells

Region		Visits	Revenue	Change
North	Q1	1,240	€ 48,200	+4.2%
	Q2	1,410	€ 52,900	+6.8%
South	Q1	980	€ 31,400	−1.1%
	Q2	1,105	€ 36,750	+2.9%
Total		4,735	€ 169,250	+3.6%

4.4 Long Table with Repeated Header

The table below is built from the attached CSV and intentionally runs across a page break. The header row must repeat, and the grid must stay aligned after the break.

id	category	structure	page	state	notes
ST-001	Typography	H1 heading	3	Stable	Section opener
ST-002	Typography	H2/H3 subheads	3	Stable	TOC level 2-3
ST-003	Typography	H4/H5 subheads	3	Stable	Deep outline
ST-004	Typography	Justified paragraph	4	Stable	Wide measure
ST-005	Typography	Bullet list	5	Stable	Nested bullets
ST-006	Typography	Numbered list	5	Stable	Two-digit items
ST-007	Typography	Block quote	6	Stable	Indented rule
ST-008	Typography	Code block	6	Stable	Monospace
ST-009	Typography	Two columns	7	Stable	Frame switch
ST-010	Typography	Unicode samples	7	Stable	Greek, Cyrillic
IM-001	Images	PNG with alpha	9	Stable	Transparent
IM-002	Images	JPEG photograph	10	Stable	Color
IM-003	Images	Grayscale JPEG	10	Stable	DeviceGray
IM-004	Images	Vector diagram	9	Stable	Large raster
IM-005	Images	200 DPI scan	12	Stable	Heavy page
TB-001	Tables	Simple grid	14	Stable	Header row
TB-002	Tables	Zebra rows	14	Stable	Alternating fill
TB-003	Tables	Merged cells	15	Stable	Colspan/rowspan
TB-004	Tables	Long table	16	Stable	Repeated header
TB-005	Tables	Numeric columns	17	Stable	Right aligned
TB-006	Tables	Rich cells	18	Stable	Nested content
TB-007	Tables	Feature matrix	19	Stable	Check marks
NV-001	Navigation	Outline level 1	21	Stable	Bookmark
NV-002	Navigation	Outline level 5	21	Stable	Nested bookmark
NV-003	Navigation	Internal link	22	Stable	GOTO
NV-004	Navigation	External URL	22	Stable	URI
AN-001	Annotations	Highlight	24	Stable	Markup
AN-002	Annotations	Underline	24	Stable	Markup
AN-003	Annotations	Strikeout	24	Stable	Markup
AN-004	Annotations	Squiggly	24	Stable	Markup
AN-005	Annotations	Sticky note	25	Stable	Popup

id	category	structure	page	state	notes
AN-006	Annotations	Free text	25	Stable	Callout
AN-007	Annotations	Shapes	26	Stable	Square/circle/line
AN-008	Annotations	Ink	26	Stable	Freehand
AN-009	Annotations	Stamp	26	Stable	Rubber stamp
AN-010	Annotations	File attachment	26	Stable	Paperclip
FM-001	Forms	Text field	28	Stable	Single line
FM-002	Forms	Multiline field	28	Stable	Comment box
FM-003	Forms	Checkbox	29	Stable	On/Off
FM-004	Forms	Radio group	29	Stable	Three options
FM-005	Forms	Combo box	30	Stable	Choice
FM-006	Forms	List box	30	Stable	Options
AT-001	Attachments	CSV data	32	Stable	Table source
AT-002	Attachments	ODS workbook	32	Stable	Calc
AT-003	Attachments	JSON manifest	32	Stable	Metadata
AT-004	Attachments	Text notes	32	Stable	Plain text
RD-001	Rendering	Transparency	34	Stable	Alpha blend
RD-002	Rendering	Gradient	34	Stable	Shading
RD-003	Rendering	A3 page	36	Stable	Mixed size
RD-004	Rendering	Rotated page	37	Stable	/Rotate 90
RD-005	Rendering	Dense small type	35	Stable	7.5 pt

Source rows: 52. The CSV and ODS copies are attached to the PDF.

4.5 Numeric Alignment and Units

Metric	Min	Median	Max	Unit	Sample
Open time	82	141	310	ms	GS262
First paint	44	67	128	ms	demo.pdf
Tile render	6.4	12.1	48.7	ms	A4 / 144 dpi
Scroll frame	7.9	11.6	19.2	ms	continuous
Memory	38	112	284	MB	tile cache
Zoom anchor drift	0.0	0.4	1.8	pt	wheel

Right-aligned numbers make the decimal points easier to compare. The unit column is deliberately narrow so that “MB” and “ms” stay on one line.

4.6 Rich Cell Content

Checklist
• line one
• line two
• line three
	`search(`
	`  query='tile',`
	`  regex=False,`
	`  case=False`
	`)`
		a | b
		1 | 2
		3 | 4

Figure 4.1 — A table containing a list, a code block and a nested table.

4.7 Comparison Matrix

Feature	PDFAR	Viewer A	Viewer B	Notes
Continuous scroll	✓	✓	—	required
Five-level outline	✓	✓	✓	test section 5
Annotation editing	✓	✓	—	section 6
AcroForm widgets	✓	—	✓	section 7
Embedded files	✓	—	—	section 8
Rotated pages	✓	✓	✓	section 9
Reflow mode	✓	—	—	PDFAR-only

5. Navigation and Links

A PDF can carry three independent navigation structures: the outline tree, link annotations and page labels. This fixture uses all three, and the printed table of contents is generated from the same headings as the outline.

5.1 Outline

Level	Example in this file	Expected behaviour
1	4. Tables	Top-level chapter
2	4.3 Merged Cells	Collapsible section
3	1.2.1 Architecture	Third-level topic
4	1.2.1.1 Rendering Pipeline	Fourth-level topic
5	1.2.1.1.1 Tile Lifecycle	Fifth-level topic

The sidebar outline should expose all five levels. Expanding a parent must not move the reading position, and clicking an entry should jump to the heading, not merely to the page.

5.2 Internal Links

Internal links are implemented as GOTO destinations. The links below were created by the layout engine, not added afterwards: jump to Tables, jump to Annotations, jump to Rendering. The back to contents link returns to the printed table of contents.

The printed TOC is also clickable. Its entries are matched to the outline destinations during post-processing, so every line with a page number is a link target — including the appendix entries labelled A-1, A-2 and so on.

5.3 External Links

Destination	URL
Project	github.com/wachin/PDF-Advanced-Reader
Author	linuxfrontier@proton.me
PyMuPDF	pymupdf.readthedocs.io
Qt for Python	doc.qt.io/qtforpython-6

External links should open in the desktop browser or mail client. The application must not try to render them as internal destinations.

5.4 Page Labels

The appendix is labelled with an **A-** prefix instead of the plain physical page number. Viewers that display page labels should show **A-1** on the first appendix page, while the outline still points to the correct physical page. This is a deliberate mismatch between the printed footer and the PDF page-label tree — both are visible.

Observation point
If the status bar shows a page label, it should change from a decimal number to A-1 when the appendix begins. The footer also switches to A-n.

6. Annotations

Annotations are separate PDF objects layered above the page content. The markup annotations below target real text; the remaining objects are placed in reserved areas so their position is predictable. Open the annotation sidebar (if available) and inspect the contents, authors and modification dates.

6.1 Text Markup

This sentence is highlighted in the PDF annotation layer.

This phrase carries an underline annotation.

~~This clause has a strike-through annotation.~~

This segment has a squiggly underline.

The four annotations were added after layout by searching for the exact sentences and attaching the markup to the resulting quads. They should move with the text when the page is zoomed or rotated.

6.2 Notes and Comments

Sticky note anchor — the note icon sits to the right of this line.

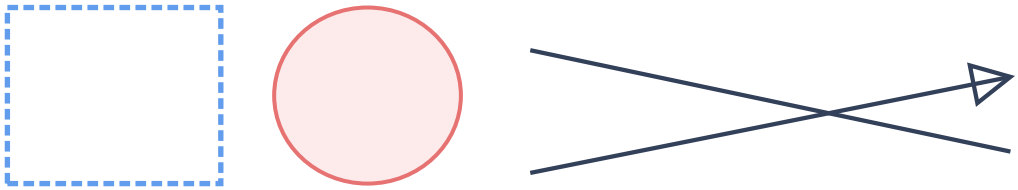


Free-text anchor — the callout box occupies the reserved area below.

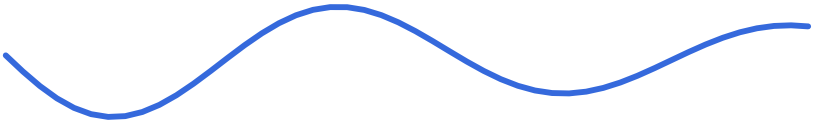
FreeText annotations embed their text in the page.

6.3 Shapes and Freehand

Shape area anchor — square, circle, line and arrow are drawn below.



Ink area anchor — a freehand stroke is drawn below.



Stamp anchor and **attachment-icon anchor**.



6.4 Annotation Inventory

Annotation	Count	Colour	Purpose
Highlight	1	Yellow	Markup a sentence
Underline	1	Blue	Markup a phrase
StrikeOut	1	Red	Markup a clause
Squiggly	1	Green	Markup a segment
Text (sticky note)	1	Amber	Popup comment
FreeText	1	Cream	Visible callout
Square / Circle	2	Blue / red	Shape annotations
Line / Arrow	2	Slate	Directional marks
Ink	1	Blue	Freehand signature
Stamp	1	Red	Rubber stamp
File attachment	1	Gray	Embedded payload

6.5 Rendering Notes

Markup annotations store quads, so a correct renderer draws them along the text baseline rather than around a bounding box. Shape annotations store a rectangle and a border style; the line annotation adds line ending styles. The stamp and file icons use appearance streams that may be supplied by the viewer if the file does not embed one.

- Markup annotations should not break text selection.
- Popup contents should be editable and persist after a round trip.
- Shape opacity is 0.35–0.55; overlapping shapes should show blending.
- The file annotation carries a small text payload independent of the document attachments.

7. Interactive Forms

AcroForm widgets are rendered by the viewer rather than baked into the page. The boxes below are intentionally left empty: the widget appearance streams provide the borders, check marks and selection dots. Use Tab to move between fields and verify that values survive a save or a form reset.

7.1 Text Fields

Single-line fields accept short values; the comment field accepts multiple lines.

Full name: Ada Lovelace

Email address: ada@example.org

Comments: Tiles render smoothly across page boundaries.

7.2 Check Boxes and Radio Buttons

Toggle at least one check box and select a different theme.

- [x] Enable tiled rendering
- [x] Show thumbnails
- [ ] Remember window state

- ( ) Light
- (•) Dark
- ( ) System

7.3 Choice Fields

Render quality: Normal (144 dpi)

Sections to include:
Typography
Images
Tables
Annotations
Forms
Attachments

7.4 Buttons

A push button has no persistent value; it should render its caption and stay interactive.

Reset form



7.5 Field Inventory

Field	Type	Value
Full name	Text	Ada Lovelace
Email address	Text	ada@example.org
Comments	Multiline text	Tiles render smoothly…
Enable tiled rendering	Check box	On
Show thumbnails	Check box	On
Remember window state	Check box	Off
Theme	Radio group	Dark
Render quality	Combo box	Normal (144 dpi)
Sections to include	List box	Tables
Reset form	Push button	—

8. Attachments and Embedded Files

Five payloads are embedded in the PDF file itself, not linked from the filesystem. They should appear in the attachments panel and remain available when the PDF is moved to another machine.

8.1 Embedded Payloads

File	Format	Purpose
table-data.csv	CSV	Source rows for the long table in section 4.4
demo-data.ods	LibreOffice Calc	Same data as a spreadsheet workbook
feature-matrix.csv	CSV	Feature support matrix
manifest.json	JSON	Build metadata and structure summary
notes.txt	Plain text	Human-readable field notes

Open **Tools > Attachments** (Ctrl+Shift+A), or switch the sidebar to the Attachments tab, then select a file to preview or extract it.

8.2 Why the Tables Are Native PDF

A spreadsheet is an excellent container for data, but it is a poor container for a table that must be read on a PDF page. If a Calc sheet were flattened into an image, the text would stop being selectable; if it were embedded as an attachment only, the table would not be visible while reading. The fixture therefore does both: every table is drawn as native PDF text, and the underlying rows are attached as CSV and ODS.

The ODS workbook was produced by LibreOffice from `table-data.csv` and then embedded unchanged. It is there to test the attachment panel, not to serve as the layout source for the PDF.

> **Design note**
> Native PDF tables preserve selection, search and copy/paste. Spreadsheet attachments preserve authoring and formulas. Keeping both representations makes the fixture useful for rendering tests and for attachment tests.

8.3 Attachments in the Reader

1. Open the Tools menu and choose Attachments, or press Ctrl+Shift+A.
2. Switch the sidebar to the Attachments tab if it is not already visible.
3. Select `demo-data.ods` and verify the size and description.
4. Extract the file and confirm that it opens in LibreOffice Calc.
5. Return to the document without losing the current page.

9. Rendering Challenges

The final section pushes the renderer into less common territory: transparency, gradients, a page larger than A4, a rotated page and very small type. These objects are cheap to draw once and expensive to get wrong.

9.1 Transparency and Gradients

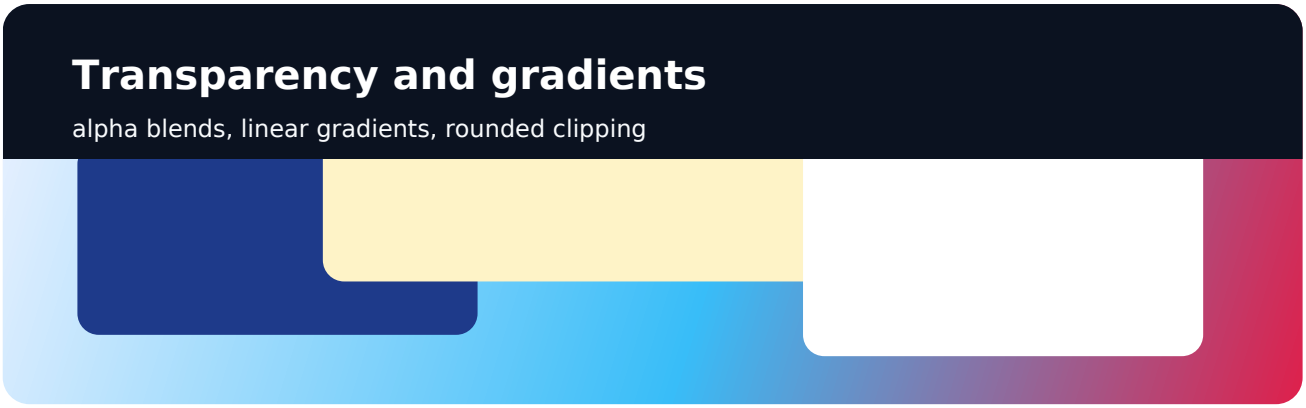


Figure 9.2 — Alpha blends over a linear gradient, drawn as PDF vectors.

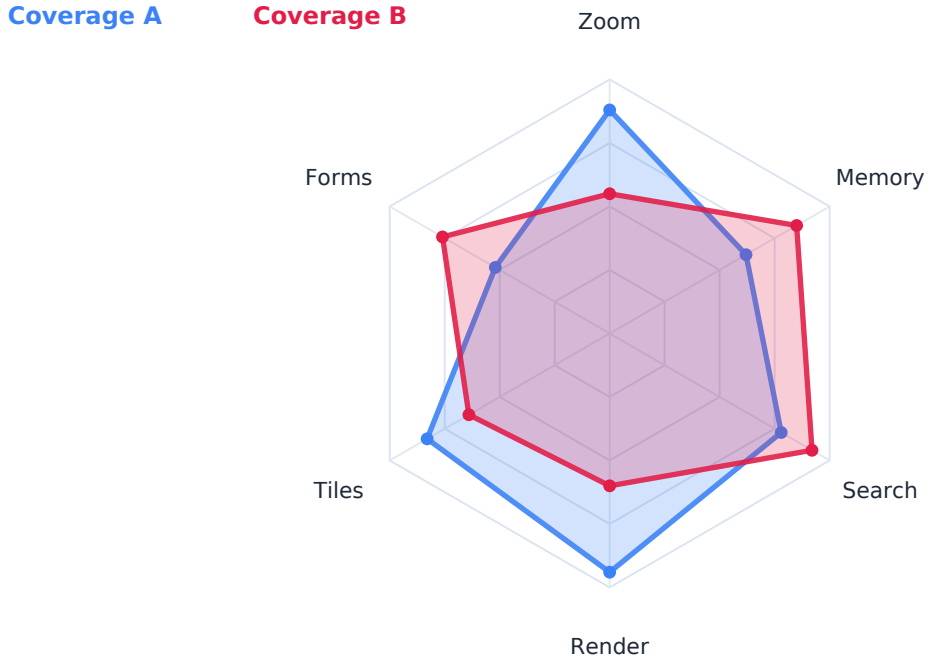


Figure 9.3 — Radar chart with two translucent series and vector axes.

9.2 Dense Text and Small Type

This paragraph is set at 7.5 pt on a 9.6 pt leading. At 100% zoom it is still legible on a high-DPI display; at 50% zoom it becomes grey texture. When reflow is enabled the text size should increase without introducing horizontal scrolling. The paragraph deliberately repeats enough words to cross several lines so that hyphenation, justification and line-breaking are exercised at a small size.

The next line contains an unbroken URL-like token: https://example.invalid/reports/2026/quarterly/summary-with-a-very-long-slug/download?format=pdf&version=2 — it should

wrap or clip predictably, not overlap the margin. The following code block has lines longer than the frame; the renderer must not expand the page to fit them.

```
GET /api/v1/documents/8f2c1a/renders?page=36&zoom=1.25&dpi=144 HTTP/1.1
Accept: image/png
X-PDFAR-Tile: 3,2;cache=warm;priority=visible;generation=17
```

A final paragraph at normal size confirms that the view returns to the regular leading after the dense block.

9.3 Large-Format Page (A3)

This page is A3 portrait (842 × 1191 pt), inserted between A4 pages. Fit-page, fit-width and the zoom presets must all use the current page geometry rather than a document-wide default. The table below is 734 pt wide and would not fit on the A4 pages.

Page	Width (pt)	Height (pt)	Aspect	Content
A4	595	842	0.707	Chapters 1–9
A3	842	1191	0.707	This spread
Rotate	595 / 842	842 / 595	swapped	Exhibit

The wide measure is also a typography test: a 734 pt line is almost twice the normal measure, so justification has more freedom and the eye needs a generous leading. The page should scale cleanly in the thumbnail panel without forcing the sidebar to resize.

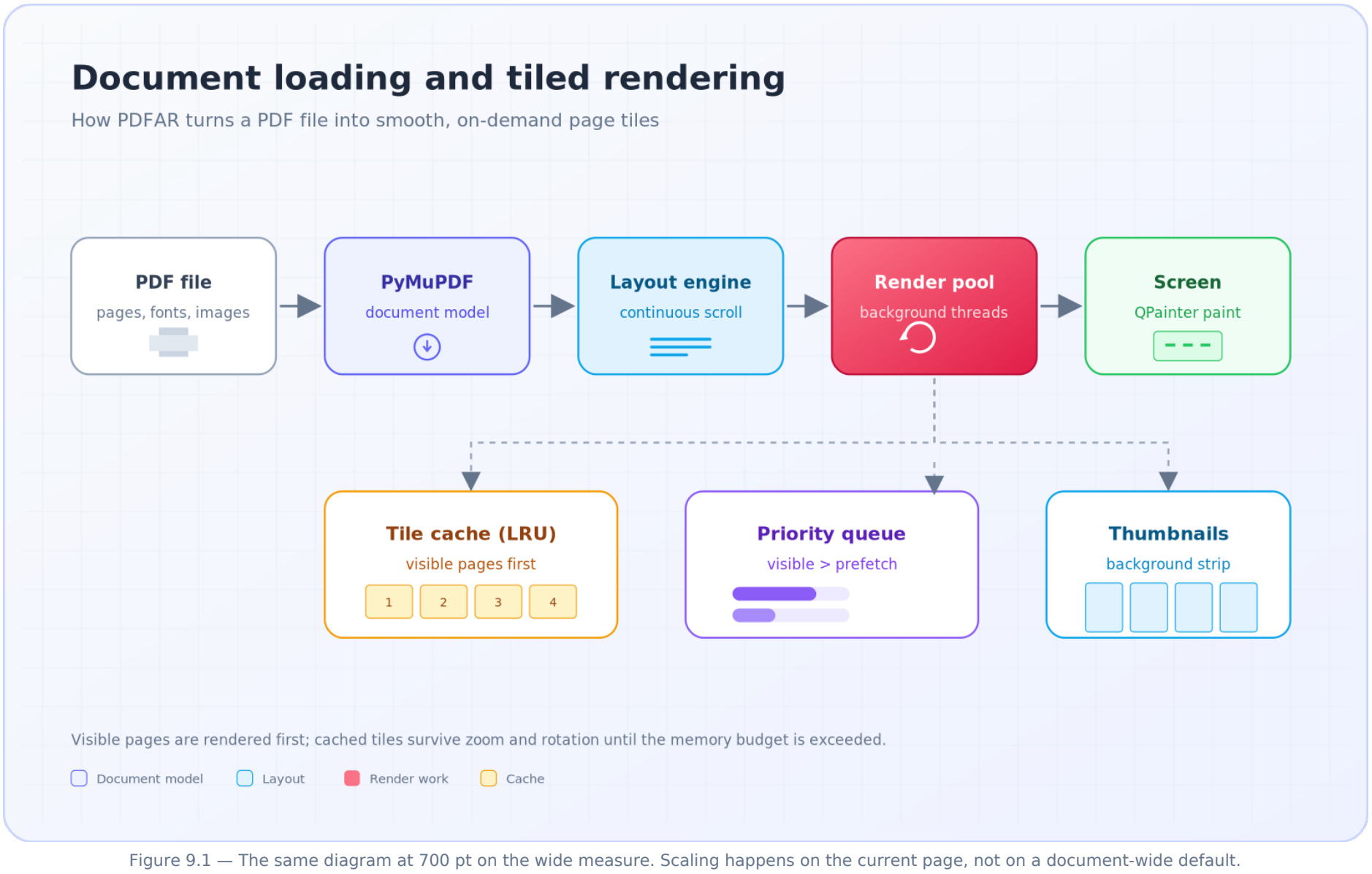


Figure 9.1 — The same diagram at 700 pt on the wide measure. Scaling happens on the current page, not on a document-wide default.

9.4 Rotated Page

The next page is stored as a portrait page with a **/Rotate 90** entry. Its media box is 595 × 842 pt, but the viewer displays it as 842 × 595 pt landscape. A correct renderer applies the rotation when painting, selecting and searching; a naïve one shows the content sideways or reports the wrong page size.

The exhibit was drawn after layout so its text can be positioned in display coordinates. Scroll past this page and then navigate back to it: the rotation must survive caching and thumbnail generation.

Element	Value
Media box	595 × 842 pt (portrait)
Rotate entry	/Rotate 90
Displayed size	842 × 595 pt (landscape)
Text layer	Rotates with the page and stays selectable
Thumbnail	Must follow the displayed orientation

- The page picker and thumbnail panel must show a landscape sheet.
- Search hits must be drawn on the visible glyphs, not the raw media box.
- Save and reopen must preserve the /Rotate entry and the page label.

9.4 ROTATED EXHIBIT

Media box 595 × 842 pt · /Rotate 90 · displayed 842 × 595 pt

Why this page exists

A page can be rotated by a PDF entry instead of by a landscape media box. The viewer must rotate painting, hit-testing, selection and thumbnails together.

If only the page pixmap is rotated, the text layer will still point to the original coordinates and selection will drift.

What to check

The page reads upright when opened.

The displayed size is 842 × 595 pt.

Text selection follows the visible glyphs.

The thumbnail matches the displayed orientation.

Going forward and back preserves the rotation.

Implementation note

The exhibit is drawn after the layout pass with display coordinates mapped into the unrotated page. The rotation is then attached to the page dictionary.

This is intentionally the last unusual page before the appendix so that page labels and the outline remain stable.

9.5 Mixed Content

After the rotated exhibit the document returns to A4. The remaining pages mix text, a small table and an image to check that the view does not retain the previous page's geometry.

Element	Expected result
Text	Normal reading measure
Table	Grid aligned to the content width
Image	Centered with a caption



Figure 9.4 — A smaller placement of the zoom diagram to test scaling after a rotated page.

10. Appendix

The appendix uses page labels with the prefix A-. The printed footer switches to A-n as well, while the outline continues to reference the physical page. This is the last group of headings, including the deepest level in the document.

10.1 Testing Checklist

10.1.1 Rendering

10.1.1.1 Scrolling

10.1.1.1.1 Tile Boundaries

Check	Pass condition
No white flashes while scrolling	Placeholders fade into tiles
Tile seams invisible	Edges match at 100% and 200%
Page boundaries clean	No clipping or overlap
Rotation applied	Exhibit reads upright in landscape
A3 page scaled	Fit page shows the full sheet

10.1.2 Interaction

Check	Pass condition
Text selection	Selection follows glyphs after rotation
Search	Hits in body, captions and scan page
Outline	Five levels expand independently
Annotations	Popup contents and colours persist
Forms	Values survive tabbing and reset
Attachments	All five payloads listed and extractable

10.2 Glossary

Term	Definition
AcroForm	PDF dictionary that describes interactive form fields.
CID font	Font whose glyphs are addressed by character IDs.
GOTO link	Link annotation that moves to a destination in the same file.
Media box	Physical page boundary before rotation is applied.
OCG	Optional content group, also known as a PDF layer.
Quad	Four-point polygon used by text markup annotations.
Tile	Rectangular render unit stored in the cache.
URI action	Link action that opens an external URL.

10.3 Revision History

Version	Date	Change
1.0	2026-10-01	Initial six-page smoke test
2.0	2026-10-23	Full fixture: images, tables, annotations, forms, attachments

10.4 Colophon

This document was generated by `tools/demo_pdf/make_demo_pdf.py` using ReportLab for layout and PyMuPDF for post-processing. The source SVG assets live next to the script and are rasterised with cairosvg (or rsvg-convert). The attached workbook was produced with LibreOffice Calc in headless mode.

Fonts: DejaVu Sans, DejaVu Serif and DejaVu Sans Mono. Colours follow the PDFAR identity: indigo and blue with a coral accent. The fixture is released under GPL-3.0-or-later, like the reader itself.

End of document · demo.pdf · generated for PDF-Advanced-Reader